Chapter 3
Time In Distributed System

Prepared By:
Er. Amrit Poudel
GCES

Why Time In Distributed System

- **External Clock Synchronization**:

  Aligns a computer's local clock to an **authoritative external source** (e.g., NTP server) to ensure consistent real-world timestamps, essential for activities like **logging** and **financial transactions**.

- **Internal Clock Synchronization**:

  Synchronizes clocks within a distributed system to maintain **consistent relative timestamps**, focusing on the **order** or **interval** between events, critical for systems like **databases** and **event-driven architectures**.

- **Network Unpredictability**:

  Latency and jitter in networks impact synchronization; algorithms must handle **variable message delays** to estimate accurate clock offsets.

- **Ignoring Relativistic Effects**:

  In most practical distributed systems, the relativistic effects of time dilation (due to speed or gravity) are negligible.

Physical Clocks

- **Physical Clock**:

  Every clock includes a **physical clock**, an electronic device counting oscillations in a crystal at a specific frequency.
- **Counter Register**:

  The count of oscillations is stored in a **counter register**, used to generate timestamps for events on the computer.
- **Clock Resolution**:

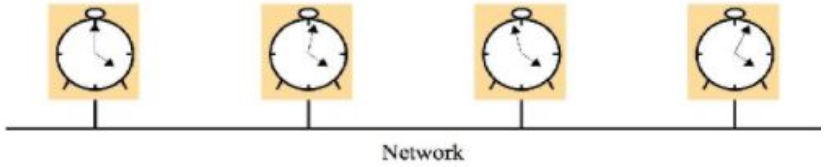
  The **clock resolution** is the interval between updates of the clock value. Two events will only have distinct timestamps if the clock resolution is **small enough**.
- **Event Order vs. Exact Time**:

  Many applications focus on the **order of events**, rather than their precise real-world timestamps

Clock Skew and Clock Drift

Clock Skew:

- The instantaneous difference between the readings of any two clocks is their skew.



Clock Drift:

- The difference in rates of two clocks
- A clock ***drift rate*** is the change in the offset(difference in reading) between the clock and a normal perfect reference clock per unit time measured by a reference clock.

Coordinated Universal Time(UTC)

Definition:

UTC is an **international timekeeping standard** based on a **24-hour format**, maintained using **atomic clocks** and the Earth's rotation.

Broadcasting:

Regularly synchronized and broadcasted via **radio stations** and **satellites**, covering large parts of the world.

Components of UTC:

- **International Atomic Time (TAI)**: Combines outputs from ~400 atomic clocks worldwide to ensure a precise tick rate.
- **Universal Time (UT1)**: Represents the Earth's rotation (astronomical/solar time) and adjusts TAI to align with the actual length of a day.

Synchronizing Physical Clocks

- In order to know at what time of day events occur at a process in the distributed system, it is necessary to synchronize the processes' clocks with an authoritative external source of time. This is called **external synchronization**.
- If a clock are synchronized with one another to a known degree of accuracy, then we can measure interval between two events occurring at different computers by appealing to their local clocks - even though they are not necessarily synchronized to an external source of time. This is **internal synchronization.**

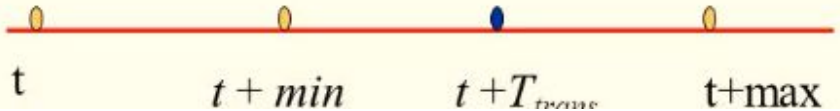
- **C_i : p_i's clock, I: an interval of real time**
- **External synchronization**
  - For a synchronization bound $D > 0$, and for a source S of UTC time, $|S(t)-C_i(t)| < D$, for $i = 1, 2, \ldots N$ and for all real times t in I
  - Clocks C_i are *accurate* to within the bound D
- **Internal synchronization**
  - For a synchronization bound $D > 0$, $|C_i(t)-C_j(t)| < D$ for $i, j = 1,2, \ldots N$, and for all real times t in I
  - Clocks C_i *agree* within the bound D

Synchronization in synchronous system

- One process sends the time *t* to the other in a message *m.*
- Then the receiver processor could set its clock to the time, $t + T_{trans}$ where T_{trans} *i*s the time taken to transmit *m* between them
- However, T_{trans} is subject to variation is unknown.
  - In general, other processes are competing for resources with the processes to be synchronized, Also, other message compete with *m* for the network
- So, there is always a minimum transmission time *min* that would be obtained if no other processes executed and no other network traffic existed.
- There is also an upper bound *max* on the time taken to transmit any message.

Synchronization in synchronous system

- **Protocol**
  - Sender: send $M(t)$
  - receiver: set time to $t + T_{trans}$
- **Bounds are know in synchronous system**
  - ❖ $\min < T_{trans} < \max$
- **So, set T_{trans} = (min+max) / 2**
  - Receiver clock = t + (min+max) / 2
- Clock skew
  - ❖ (max – min) / 2



Cristian's method of synchronizing clocks

- A time server is used to synchronize time externally
- A time server uses a device to receive signals from the source of UTC
- Upon request, the time server process *S*, supplies the time according to its clock

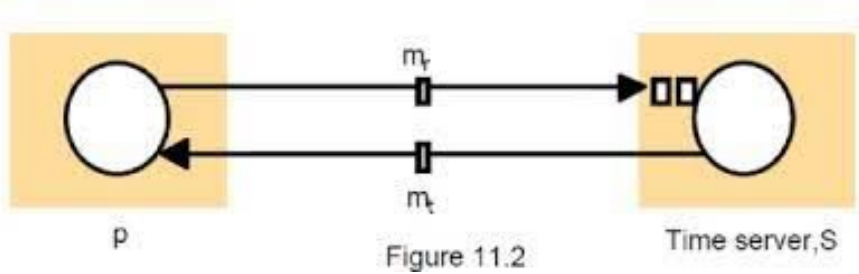


Figure 11.2

- Process p records the total round trip time T_{round} taken to send the request m_r and receive the reply m_t

Cristian’s method for synchronizing clocks

- The method achieves synchronization only if the observed round trip times between client and server are sufficiently short compared with the required accuracy
- A simple estimate of the time to which *p* should set its clock is $t + T_{round}/2$

Cristian's method for synchronizing clocks

- *If min is known, then the accuracy is as follows:*
- *The earliest point at which S could have placed the time in* m_t *was min after p dispatched* m_r.
- *The latest point at which it could have done this was min before* m_t *arrived at p.*
- *The time when reply message arrives is in the range*

$$[t + min,\ t + T_{round} - min]$$

- *The width of this range is* $T_{round} - 2min$ *, so the accuracy is*

$$\pm(T_{round} \ / \ 2 - min)$$

- *Variability can be dealt by making several requests to S and taking the minimum value of* T_{round}.

Discussion of Cristian's algorithm

- ***Single time server*** *might fail and thus render synchronization temporarily impossible.*
  - *Cristian suggested, that time should be provided by a group of synchronized time servers, each with a receiver for UTC time signals.*
  - *For example, a client could multicast its request to all servers and use only the first reply obtained.*
- ***Faulty time server****: These problems were beyond the scope of the work described by Cristian, which assumes that sources of external time signals are self-checking.*

The Berkeley Algorithm

- Gusella and Zatti describe an algorithm for internal synchronization.
- A coordinator computer is chosen to act as the *master*.
- This computer periodically polls the other computers whose clocks are to be synchronized, called *slaves*.
- The slaves send back their clock values to it.
- The master estimates their local clock times by observing the round-trip times (similar to cristian's algorithm) and it averages the values obtained(including its own clock reading)
- Instead of sending the updated current time back to the other computers, the master sends the amount by which each individual slave's clock requires adjustment.
- This can be a positive or negative value.

The Berkeley Algorithm

- The Berkeley algorithm eliminates readings from faulty clocks.
- A subset is chosen of clocks that do not differ from one another by more than a specified amount, and the average is taken of readings from only these clocks.
- If the master fail, then another can be elected.
- If the master clock fails and the election of a new master takes too long, the synchronization process halts, and the difference between clocks may grow unbounded.

The Network Time Protocol

- Cristian’s method and the Berkeley algorithm are intended primarily for use within intranets.
- The Network Time Protocol(NTP) defines an architecture for a time service and a protocol to distribute time information over the Internet

NTP’s chief design aims and features

Accurate Synchronization:

- Ensures clients across the Internet synchronize their clocks accurately to **Coordinated Universal Time (UTC)**.

Reliability and Fault Tolerance:

- Utilizes **redundant servers** and **alternative network paths** to ensure continuous service.
- Automatically reconfigures servers to maintain synchronization even if some become unreachable.

Frequent Resynchronization:

- Clients resynchronize regularly to minimize clock drift caused by hardware imperfections.

Security and Interference Protection:

- Implements **authentication** to verify that timing data comes from trusted sources.
- Validates incoming requests and ensures responses reach only authorized clients, preventing spoofing or man-in-the-middle attack

Network Time Protocol(NTP)

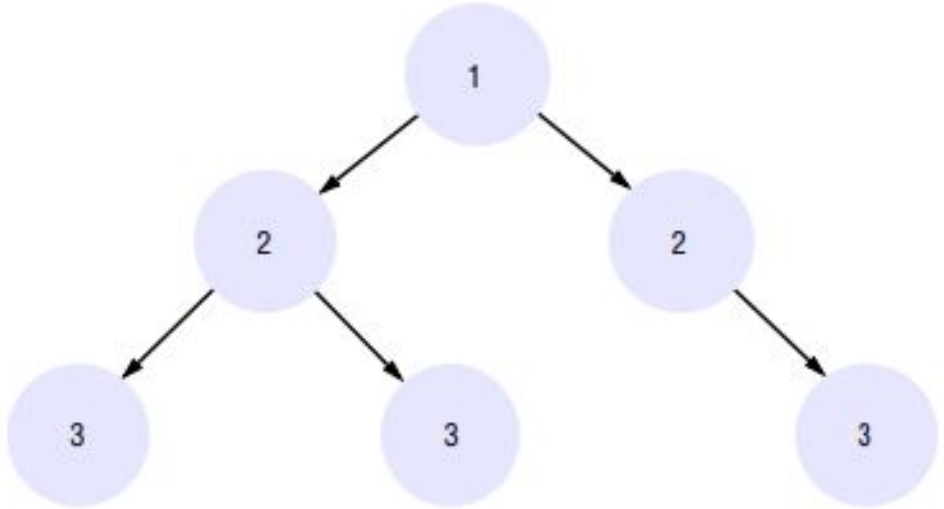
The **Network Time Protocol (NTP)** ensures precise time synchronization across the Internet through a distributed network of servers organized in a **logical hierarchy** known as the **synchronization subnet**.

Server Hierarchy:

1. **Primary Servers:**
    - Directly linked to trusted time sources such as radio clocks or atomic clocks, which receive **Coordinated Universal Time (UTC)**.
2. **Secondary Servers:**
    - Indirectly synchronized with primary servers, providing accurate time distribution to clients and other servers in the network.

This hierarchical structure enables efficient and reliable propagation of accurate time across vast networks, ensuring synchronization at all levels.

Synchronization Subnet



Arrows denote synchronization control, numbers denote strata.

Synchronization Subnet

- Primary servers occupy stratum 1: they are at the root.
- Stratum 2 servers are secondary servers that are synchronized directly with the primary servers;
- stratum 3 servers are synchronized with stratum 2 servers, and so on.
- The lowest-level (leaf) servers execute in users' workstations.
- The clocks belonging to servers with high stratum numbers are liable to be less accurate than those with low stratum numbers, because errors are introduced at each level of synchronization
- NTP also takes into account the total message round-trip delays to the root in assessing the quality of timekeeping data held by a particular server

Synchronization Subnet

- The synchronization subnet can reconfigure as servers become unreachable or failures occur
  - If, for example, a primary server’s UTC source fails, then it can become a stratum 2 secondary server.

NTP Server Synchronization

- NTP servers synchronize with one another in one of three modes:
  - Multicast,
  - Procedure-call
  - Symmetric mode

MultiCast Mode of NTP Servers Synchronization

Multicast mode in the **Network Time Protocol (NTP)** is designed for high-speed **Local Area Networks (LANs)** to efficiently synchronize time across multiple systems.

Key Features:

1. **Time Distribution:**
   - One or more servers periodically multicast the time to all computers on the LAN.
   - Client systems adjust their clocks based on the received time, assuming a minimal network delay.
2. **Efficiency:**
   - Ideal for scenarios where multiple systems need time synchronization with minimal configuration or communication overhead.
3. **Accuracy:**
   - Provides **relatively low accuracy**, as the small delay assumption introduces some error.
   - However, this level of accuracy is sufficient for many LAN-based applications.

Multicast mode simplifies time synchronization in environments where high precision is not critical, but consistent timing across systems is required.

Procedural Mode of NTP Servers Synchronization

- Procedure-call mode is similar to the operation of Cristian’s algorithm,
- In this mode, one server accepts requests from other computers, which it processes by replying with its timestamp (current clock reading).
- This mode is suitable where higher accuracies are required than can be achieved with multicast, or where multicast is not supported in hardware.

Symmetric Mode of NTP Servers Synchronization

Intended Use: Symmetric mode is used by servers that supply time information in Local Area Networks (LANs) and in higher levels of the synchronization subnet for achieving high accuracy.

Bidirectional Communication: A pair of servers operate in symmetric mode by exchanging messages that carry timing information.

Shared Responsibility: Both servers in symmetric mode are equally responsible for timekeeping, ensuring that neither server is the sole time provider.

Improved Accuracy: By continuously exchanging timing data, the synchronization between the servers improves over time, resulting in higher precision.

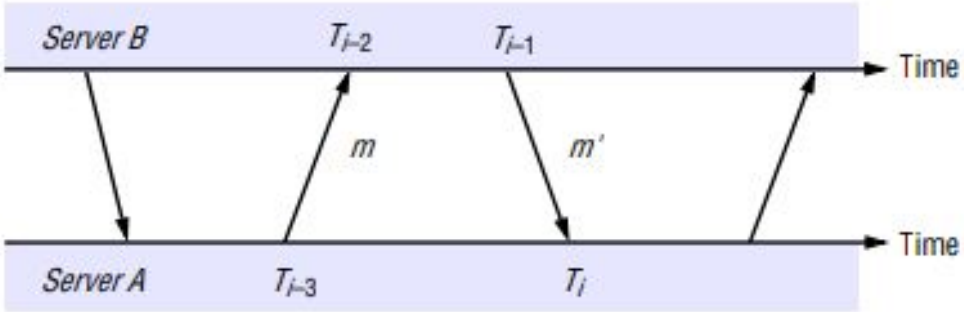
Association Between Servers: The timing information is retained as part of an association between the two servers, which helps refine synchronization accuracy over time.

High Precision: Symmetric mode is specifically designed to achieve the highest levels of time accuracy in networks where precision is critical.

For each pair of messages sent between two servers the NTP calculates an offset

- o_i , is an estimate of the actual offset between the two clocks, and a delay
- d_i , is the total transmission time for the two messages.
- If the true offset of the clock at B relative to that at A is ***o***, and if the actual transmission times for m and m' are **t** and **t'**

Messages exchanged between a pair of NTP peers



$$T_{i-2} = T_{i-3} + t + o \text{ and } T_i = T_{i-1} + t' - o$$

This leads to:

$$d_i = t + t' = T_{i-2} - T_{i-3} + T_i - T_{i-1}$$

and:

$$o = o_i + (t' - t)/2, \text{ where } o_i = (T_{i-2} - T_{i-3} + T_{i-1} - T_i)/2$$

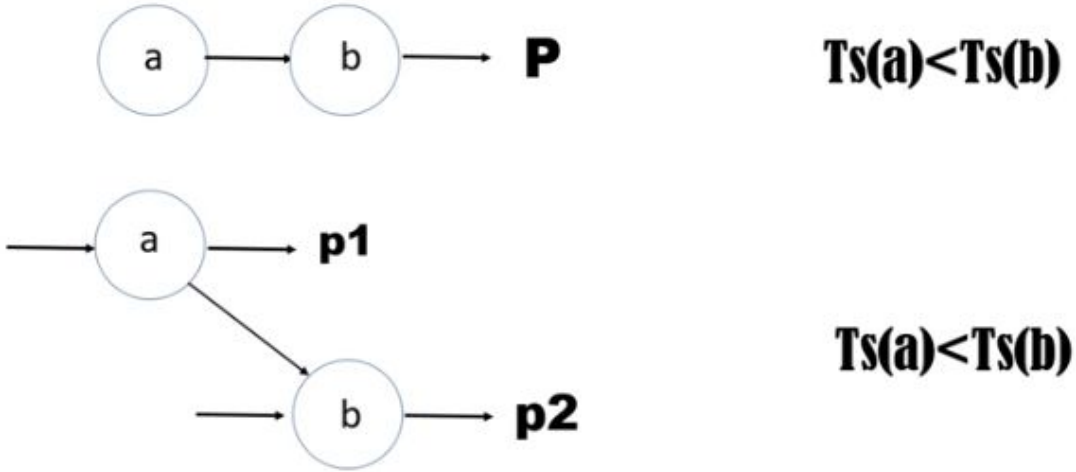
Logical time and logical clocks

- since we cannot synchronize clocks perfectly across a distributed system, we cannot in general use physical time to find out the order of any arbitrary pair of events occurring within it.
- Logical clock refer to implementing a protocol on all machines within the distributed system so that the machines are able to maintain consistent ordering of events within some virtual timestamps.

Causal Ordering or Potential Causal Ordering

Also known as:

HAPPEN BEFORE RELATIONSHIP



Properties Derived from Happen Before Relationship –

- **Transitive Relation –**

  If, TS(A) <TS(B) and TS(B) <TS(C), then TS(A) < TS(C)
- **Causally Ordered Relation –**

  a->b, this means that a is occurring before b and if there is any changes in a it will surely reflect on b.
- **Concurrent Event –**

  This means that not every process occurs one by one, some processes are made to happen simultaneously i.e., A || B. Example: if A is not happening before B and B is not happening before A. It means that A and B are happening concurrently.

Lamport's Logical Clock

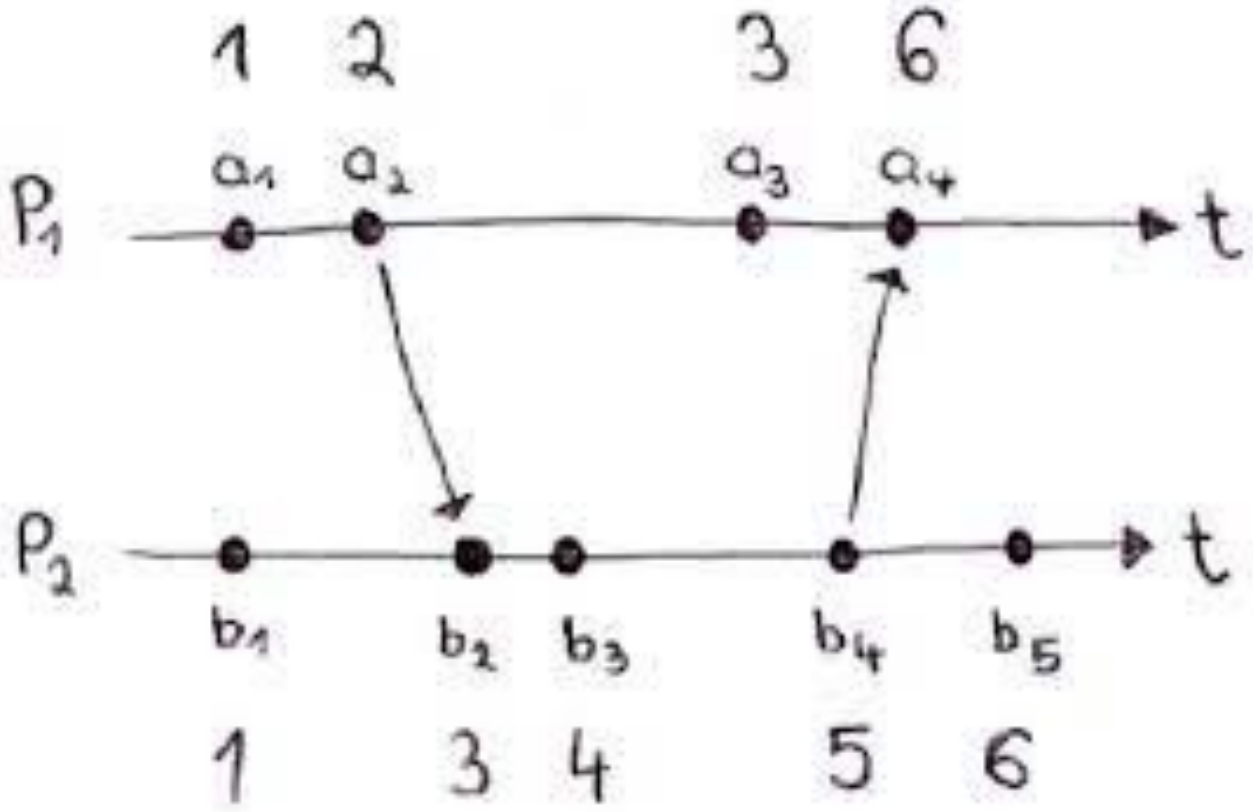
- Lamport's logical clocks capture the happen-before relation:
- Each process **pi** keeps its own clock **Li**

The rules to update Li are:

LC1: Li is incremented *before* each event at p_i: $L_i := L_i + 1$.

LC2a: When a process p_i sends a message m, it piggybacks on m the value $t := L_i$.

LC2b: On receiving (m,t), a process p_j computes $L_j := \max(L_j, t)$ and then applies LC1 before timestamping the event *receive(m)*.


1 2 3 6
a_1 a_2 a_3 a_4
P_1
t
P_2
t
b_1 b_2 b_3 b_4 b_5
1 3 4 5 6

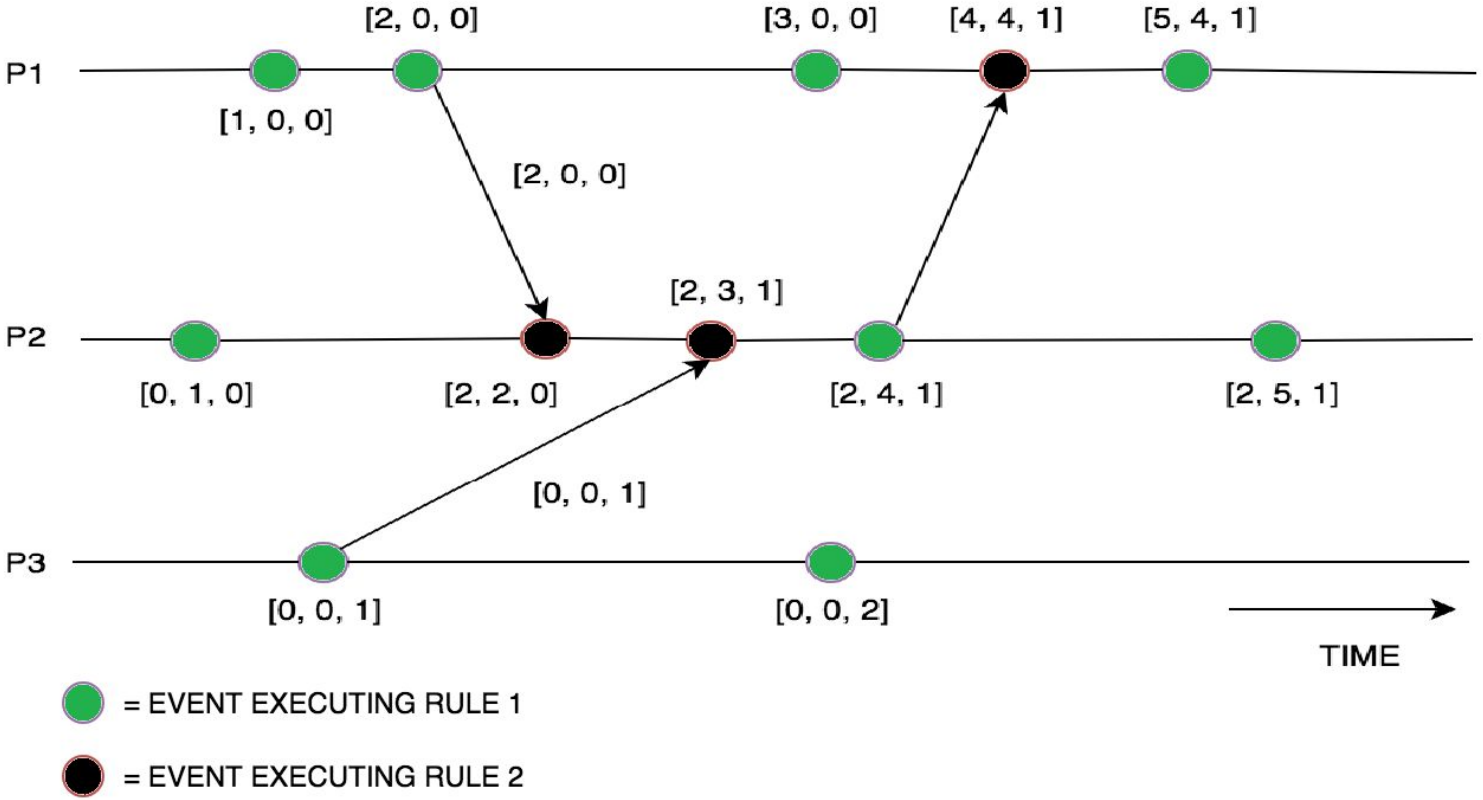
Vector Clock

Vector Clocks represent an extension of Lamport Timestamps in that they guarantee the strong clock consistency condition which (additionally to the clock consistency condition) dictates that if one event's clock comes before another's, then that event comes before the other, i.e., it is a two-way condition.

Vector Clock

- Initially, all the clocks are set to zero.
- Every time, an Internal event occurs in a process, the value of the processes's logical clock in the vector is incremented by 1
- Also, every time a process sends a message, the value of the processes's logical clock in the vector is incremented by 1.
- Every time, a process receives a message, the value of the processes's logical clock in the vector is incremented by 1, and moreover, each element is updated by taking the maximum of the value in its own vector clock and the value in the vector in the received message (for every element).

Vector Clock



Global State in Distributed Systems:

- **Definition**: A global state is the complete state of the entire distributed system at a particular point in time. Since distributed systems involve multiple independent nodes (computers or processes) that work concurrently, the global state consists of the local states of each of these nodes, including their variables, buffers, message queues, and other resources.
- **Challenge**: In a distributed system, it's difficult to have a single, consistent view of the global state because each node operates independently, potentially with different clocks, and without a centralized controller. The system's global state can be influenced by network delays, crashes, or other asynchronies between the nodes.
- **Global State Components**:
    - **Local states**: The state of each individual node (e.g., process variables, data in memory).
    - **Messages in transit**: Messages that are sent but have not yet been received by their intended destination. This is important because an in-transit message can change the state of the receiving node.
- **Use cases of Global State**:
    - **Consistency checking**: Ensuring that the system is in a valid state and that no two operations conflict.
    - **Fault tolerance**: After a failure, it’s crucial to know the global state to restore the system to a consistent point.
    - **Coordination**: In distributed algorithms, a consistent global state is often required for making global decisions, such as consensus or leader election

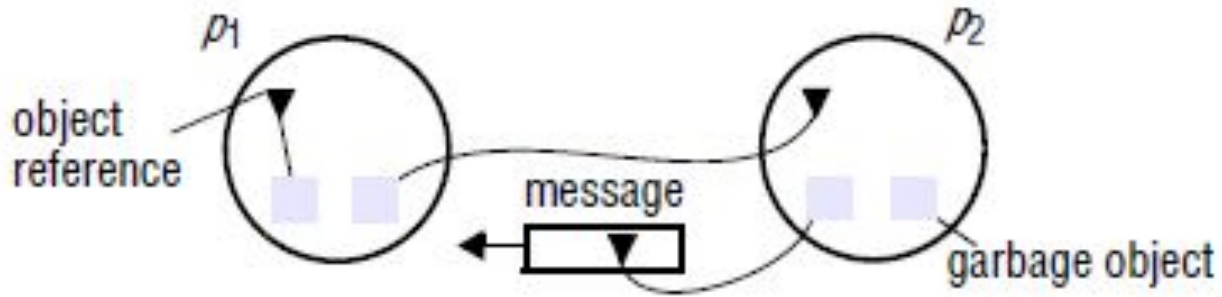
Global States

A few examples to construct a global view of the system state are:

- **Distributed garbage collection:**
- **Distributed deadlock detection:**
- **Distributed termination detection**
- **Distributed debugging:**
  -

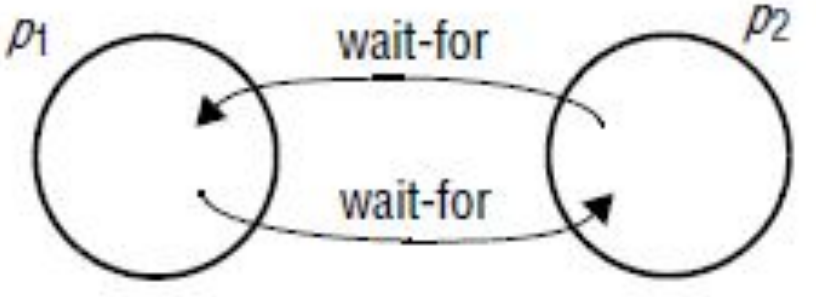
Distributed garbage collection:

- Are there references to an object anywhere in the system?
- References may exist at the local process, at another process, or in the communication channel.

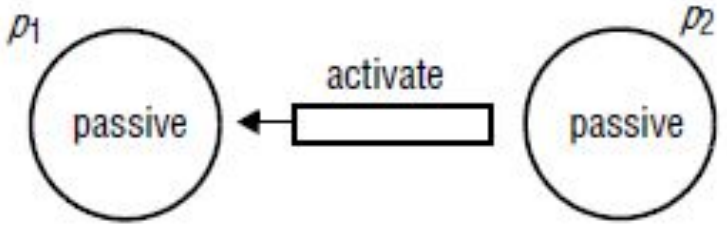


Distributed deadlock detection

- Is there a cycle in the graph of the "waits for" relationship between processes?



Distributed termination detection:



Has a distributed algorithm terminated?

- a process is either active or passive
- a passive process is not engaged in any activity of its own but is prepared to respond with a value requested by the other.
- Suppose we discover that p1 is passive and that p2 is passive
- Seeing this, we may not conclude that the algorithm has terminated,
- consider the following scenario: when we tested p1 for passivity, a message was on its way from p2 , which became passive immediately after sending it. On receipt of the message, p1 became active again – after we had found it to be passive. The algorithm had not terminated.

Snapshot in Distributed System

- **Definition**: A snapshot is a recorded state of a distributed system at a particular moment. It's a way to capture a "snapshot" of the global state, typically by recording the local states of each node and the state of messages in transit. A snapshot allows a system to "freeze" its current state for analysis, recovery, or debugging.
- **Key Concept**: A snapshot needs to account for **messages in transit**, because simply recording the state of the nodes doesn't give the full picture. If messages are in the process of being sent, they could affect the state of the system once they arrive, so it’s necessary to account for them as part of the snapshot.
- **Chandy-Lamport Algorithm**: One of the most well-known algorithms for capturing consistent snapshots in a distributed system is the **Chandy-Lamport algorithm**. It is used to take consistent snapshots without stopping the system, meaning that processes can continue running while a snapshot is being taken. The algorithm ensures that the snapshot contains the local states of all processes and the state of all messages in transit without causing inconsistencies.
- **Snapshot Components**:
    - **Local state of each process**: The current state of all processes in the system.
    - **Channels**: The messages that are in transit at the time of the snapshot. It records whether a message is being sent and has yet to be received, or if it has been completely delivered.
- **Why Take Snapshots?**:
    - **Rollback recovery**: In case of failures, a snapshot allows the system to revert to a previously recorded state, helping to recover from errors or inconsistencies.
    - **Debugging and analysis**: A snapshot provides a point-in-time view that can be analyzed to track issues, such as deadlocks or unexpected behavior.
    - **Global consistency**: In distributed databases or coordination systems, a snapshot is used to maintain a consistent state across the system, even when operations are happening concurrently.

Marker Sending Rule for process i

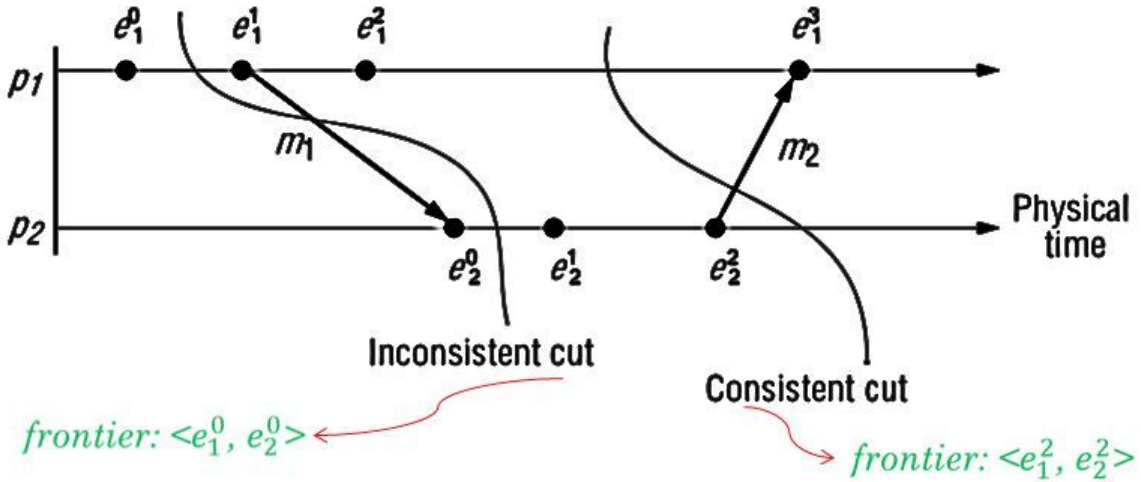
- Process i records its state.
- For each outgoing channel C on which a marker has not been sent, i sends a marker along C

Marker Receiving Rule for process j

On receiving a marker along channel C:

- if j has not recorded its state then
  - Record the state of C as the empty set
  - Follow the "Marker Sending Rule"
- else
  - Record the state of C as the set of messages received along C after j 's state was recorded and before j received the marker along C

Consistent cuts



- *A cut C is **consistent** if, for each event it contains, it also contains all the events that happened-before that event:*

  For all events $e \in C$, $f \longrightarrow e$ then $f \in C$

- *The leftmost cut is inconsistent. This is because at p_2 it includes the receipt of the message m_1, but at p_1 it does not include the sending of that message. This is showing an 'effect' without a 'cause'.*